MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

Department of Information and Communication Technology

PHISHING WEBSITE DETECTOR SYSTEM

Final Capstone Project Proposal and Prototype (Part-I)

Student Name: Monira Akter Mahin

Student ID: IT24017

Course Code: ICT-3107

Course Name: Software Engineering and Project Management

Submitted by

Monira Akter Mahin

Department of ICT

Supervised by

Professor Dr. Ziaur Rahman

Department of ICT

October 28, 2026

Contents

1 Introduction 3

1.1 Background and Problem Statement 3

1.2 Objectives 3

1.3 Scope 3

1.4 Stakeholders 3

2 Literature Review / Related Work 3

3 Requirements 3

3.1 Functional Requirements 3

3.2 Non-Functional Requirements 3

4 Technology and Tools 4

5 Software Process Model 4

6 Project Schedule 4

6.1 Gantt Chart (16 weeks) 4

6.2 Milestones and Deliverables 5

7 Risk Analysis 5

8 Cost and Resources 5

9 Testing and Quality Plan 5

10 UML Models 6

10.1 Data Flow Diagram 6

10.2 Use Case Diagram 6

10.3 Activity Diagram (Scan a URL) 7

10.4 Entity Relationship Diagram 7

10.5 Class Diagram 8

10.6 Object Diagram 8

11 Prototype Interface 8

12 Conclusion and Future Work 9

References 9

1 Introduction

1.1 Background and Problem Statement

Phishing is one of the most common cyber-attacks. Blacklists are slow to update, and ordinary users cannot reliably judge whether a link is safe. A lightweight tool that evaluates a URL instantly can reduce the number of victims.

1.2 Objectives

- Build a detector that classifies a URL as legitimate or phishing with a risk score (0–100).
- Reach at least 90% accuracy on a public labelled dataset (e.g. PhishTank / UCI Phishing).
- Provide an easy GUI and a stored scan history.
- Deliver complete documentation, UML models and tests.

1.3 Scope

In scope: URL-based analysis, risk scoring, scan history, reporting suspicious URLs, CSV export. **Out of scope (this version):** email scanning, browser extension, page-content/screenshot analysis (future work).

1.4 Stakeholders

Students/end users, analyst/administrator, course supervisor.

2 Literature Review / Related Work

Existing approaches: (1) blacklist services (Google Safe Browsing, PhishTank), (2) heuristic rules on URL structure, (3) machine learning on URL/HTML features (Random Forest, SVM). PhishGuard starts with heuristics and upgrades to a Random Forest classifier trained on public data.

3 Requirements

3.1 Functional Requirements

ID	Requirement
FR1	The user can enter a URL and request a scan.
FR2	The system extracts features (length, IP host, '@', dots, hyphens, HTTPS, keywords, shorteners).
FR3	The system classifies the URL and displays label and risk score.
FR4	Every scan is stored with time stamp and can be viewed later.
FR5	The user can report a URL as phishing/legitimate (feedback).
FR6	The administrator can retrain the model and manage the blacklist.
FR7	History can be exported to CSV.

3.2 Non-Functional Requirements

Performance: result in $< 2\,\text{s}$. Usability: single-screen GUI. Reliability: no crash on malformed URLs. Portability: Windows/Linux. Security: no visiting of the suspicious URL (static analysis only). Maintainability: modular classes, unit tests $\geq 70\%$ coverage.

4 Technology and Tools

Python 3, Tkinter (GUI), scikit-learn, SQLite, pytest, Git/GitHub, LaTeX (Overleaf), PlantUML/TikZ.

5 Software Process Model

The **Agile Incremental (Scrum-style) model** is used. Requirements are small and may evolve (e.g. moving from rules to ML), so four 2–3 week sprints each deliver a working increment, with a review at the end of every sprint.

Sprint	Weeks	Increment
0	1–4	Planning, requirements, UML design
1	5–6	URL feature extractor
2	7–9	Rule-based → ML classifier
3	10–12	GUI + database + history
4	13–16	Testing, documentation, release

Table 2: Sprint plan

6 Project Schedule

6.1 Gantt Chart (16 weeks)

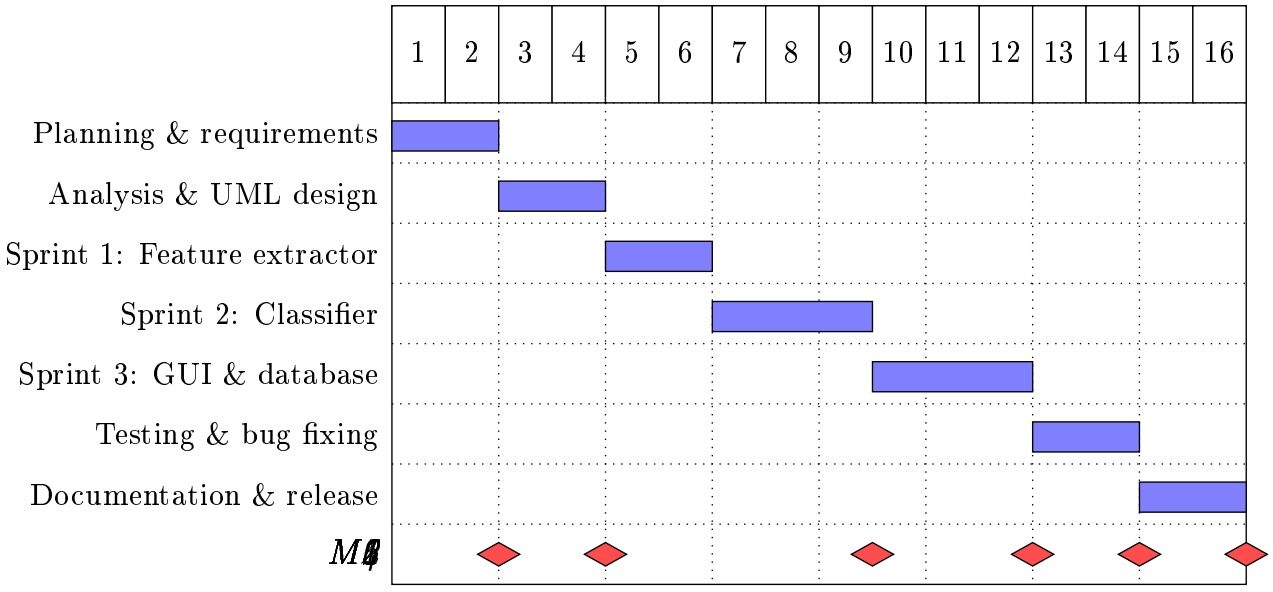


Figure 1: Project Gantt chart

6.2 Milestones and Deliverables

ID	Week	Milestone	Deliverable
M1	2	Requirements approved	Project proposal, SRS
M2	4	Design complete	UML diagrams, DB schema, UI mock-up
M3	9	Detection engine ready	Extractor + classifier, accuracy report
M4	12	Working prototype	GUI application, SQLite history
M5	14	Testing complete	Test plan, test report, bug log
M6	16	Final release	Source code (GitHub), user manual, final report, presentation

Table 3: Milestones and deliverables

7 Risk Analysis

Risk	Prob.	Impact	Mitigation
Dataset is outdated/imbalanced	M	H	Use several datasets, balance classes
Low model accuracy	M	H	Feature selection, try several models
Schedule slip	M	M	Time-boxed sprints, buffer in weeks 15–16
False positives annoy users	H	M	Adjustable threshold, user feedback
Tool/library issues	L	M	Pin versions, use virtual environment

Table 4: Risk register

8 Cost and Resources

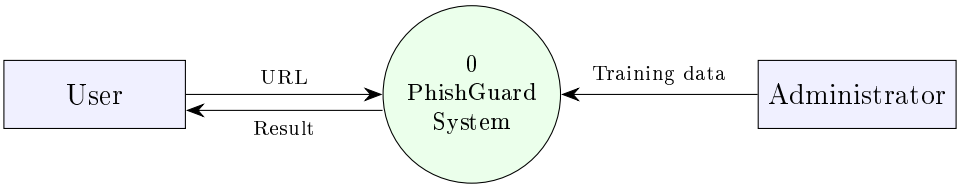
Single developer (about 10 h/week, 160 h total). All tools are free/open source, so the monetary cost is near zero; the main cost is effort. Hardware: one laptop, 8 GB RAM.

9 Testing and Quality Plan

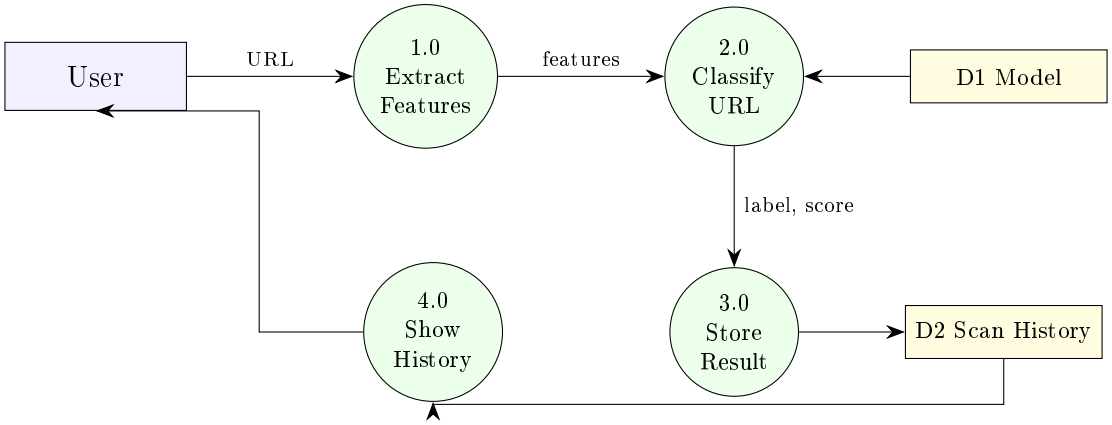
Unit tests (pytest) for each class, integration tests (scan $\rightarrow$ DB), a labelled-dataset evaluation (accuracy, precision, recall, F1), and usability checks with 5 peers. Code reviewed via GitHub pull requests.

10 UML Models

10.1 Data Flow Diagram



(a) Level 0 – Context diagram



(b) Level 1 DFD

Figure 2: Data Flow Diagrams

10.2 Use Case Diagram

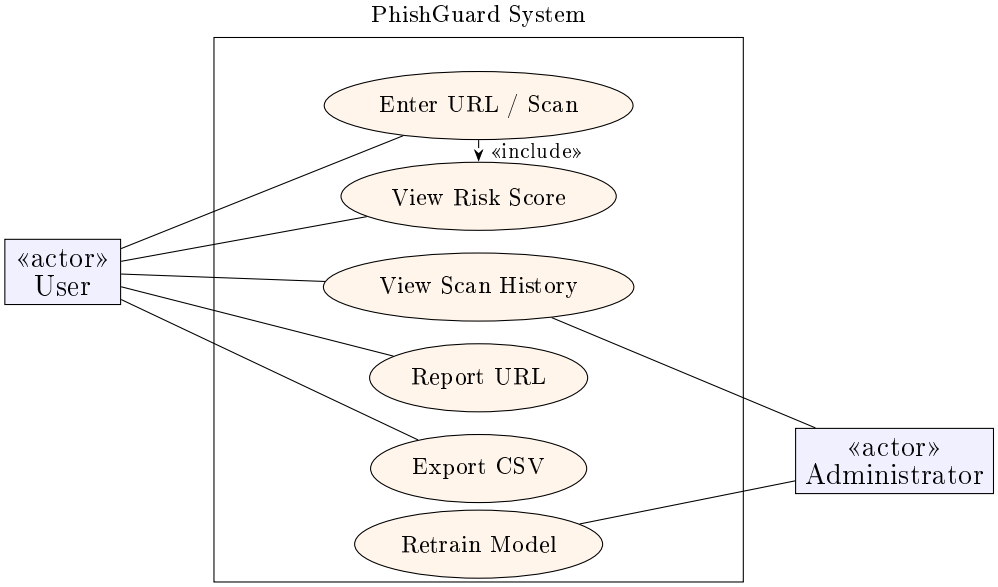


Figure 3: Use Case Diagram

10.3 Activity Diagram (Scan a URL)

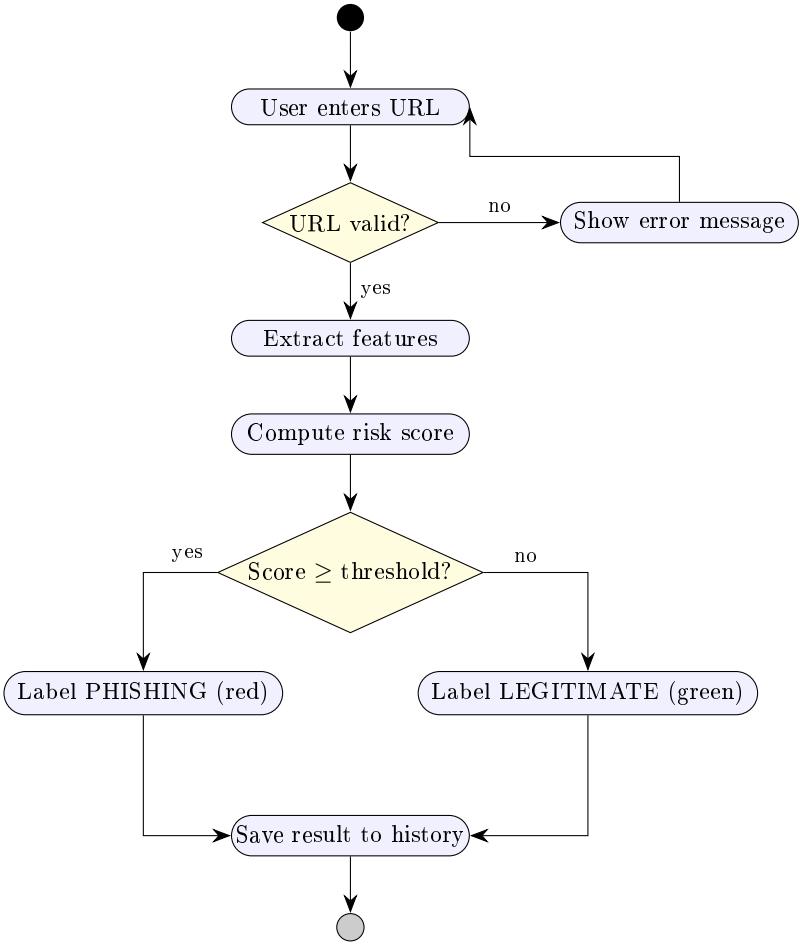


Figure 4: Activity Diagram

10.4 Entity Relationship Diagram

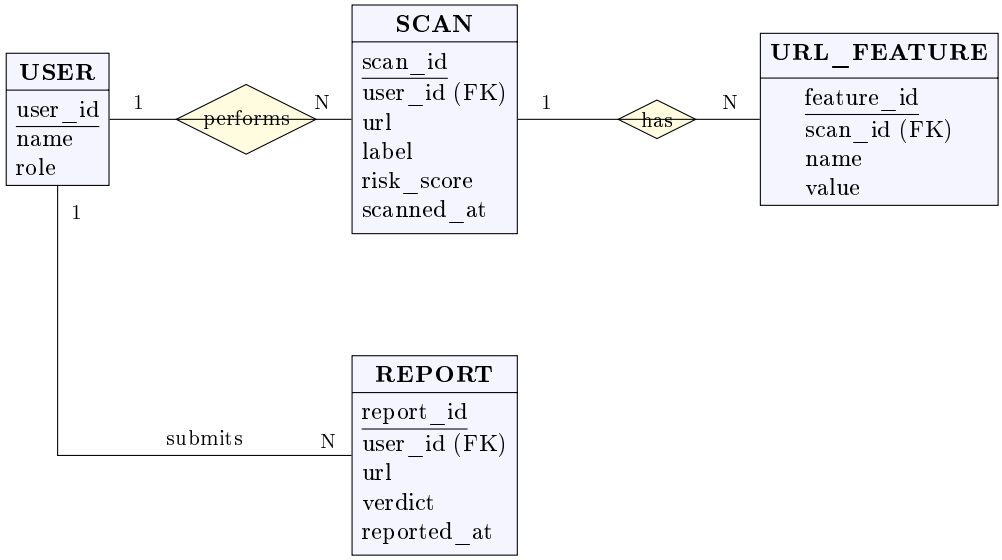


Figure 5: Entity Relationship Diagram

10.5 Class Diagram

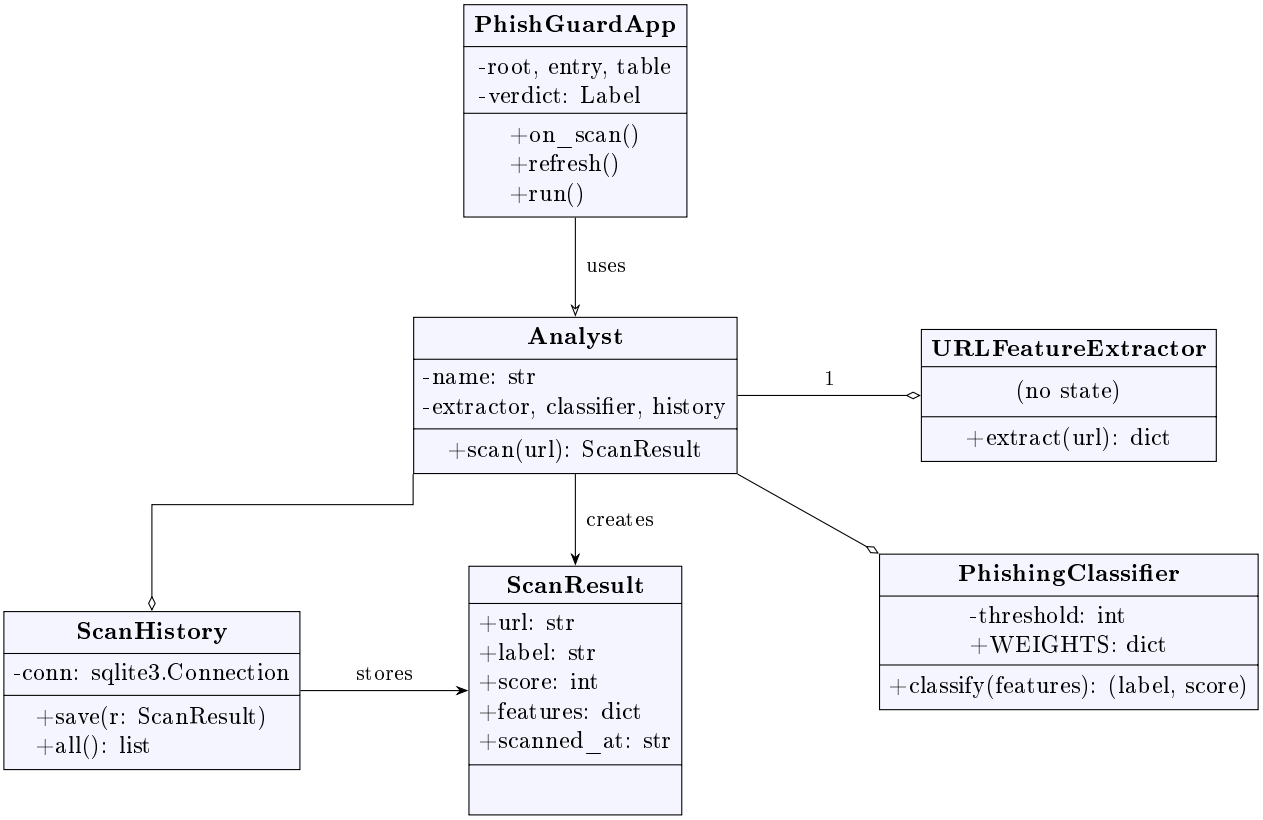


Figure 6: Class Diagram (6 classes implemented in `src/phishguard/main.py`)

10.6 Object Diagram

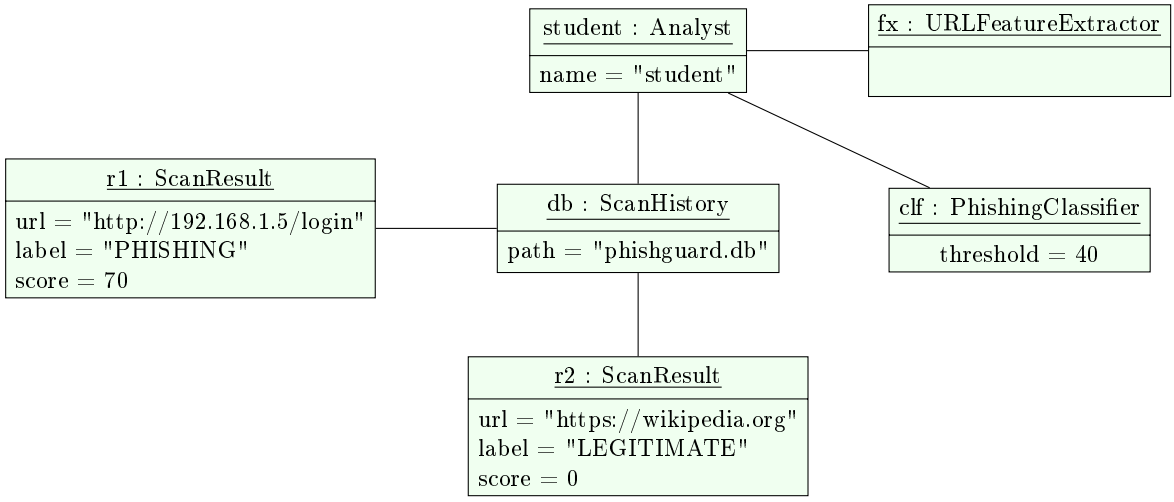


Figure 7: Object Diagram (runtime snapshot after two scans)

11 Prototype Interface

The initial interface (Tkinter) is implemented with six classes from the class diagram (`PhishGuardApp`, `Analyst`, `URLFeatureExtractor`, `PhishingClassifier`, `ScanResult`, `ScanHistory`). It has a

URL box, a *Scan URL* button, a coloured verdict (red/green) with risk score, and a history table.

```
analyst = Analyst("student", URLFeatureExtractor(),
                  PhishingClassifier(), ScanHistory("phishguard.db"))
PhishGuardApp(analyst).run()
```

Run: `cd src/phishguard && python main.py`.

12 Conclusion and Future Work

PhishGuard delivers a practical first line of defence against phishing links. Future work: ML model with HTML/WHOIS features, browser extension, e-mail scanning, REST API.

References

[1] PhishTank, https://phishtank.org.

[2] UCI Machine Learning Repository, Phishing Websites Dataset.

[3] I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2015.

[4] R. Pressman, *Software Engineering: A Practitioner's Approach*, McGraw-Hill.